ARRIS Company Admin â€” Complete Documentation

PDF export â€” combined from markdown sources in this folder.

ARRIS Company Admin — Functional Documentation

This folder documents the **Company Admin** product surface: navigation from `CompanyAdminSidebar`, related routes, UI patterns, and how modules connect.

PDF export (single bundle)

- **File:** ARRIS-Company-Admin-Documentation.pdf — one printable PDF (README + all module files).
- **Regenerate** (requires Google Chrome and Node.js `npx`):

```
cd docs/company-admin
./generate-pdf.ps1
```

Mermaid diagrams in the markdown appear as **code blocks** in the PDF (they are not rendered as graphics).

Who this is for

- **Developers** onboarding to the Next.js frontend (`frontend/`)
- **Product / BA** stakeholders mapping screens to business capabilities
- **QA** planning regression around list → view → edit flows

How the shell works

Piece	Path / component	Role
Company chrome (sidebar + navbar)	`CompanyAdminDashboardLayout`	Wraps `/company-admin/*` via `app/company-admin/layout.tsx`. Also imported by pages **outside** that prefix (e.g. `/leads`, `/projects-inventory`, `/work-orders`) so the same admin shell appears.
Sidebar definition	`src/components/layout/CompanyAdminSidebar.tsx`	Hub rows, flyouts (create / drafts / history shortcuts), `navItemIsActive` rules.

Piece	Path / component	Role
Hover / pin collapse	`useSidebarHoverCollapse`	Rail width and hover-expand behavior.
Global history UI	`/company-admin/history-logs` + `?module=`	Filterable audit trail; modules align with `HistoryModule` in `src/lib/historyLogs/types.ts`.

Tech stack (frontend)

- **Next.js App Router**, `'use client'` on interactive pages
- **Tailwind CSS 4**, shared `cn()` helper
- **State**: feature **singleton stores** under `src/lib/*Store.ts` (in-memory mock data; refresh clears unless persisted elsewhere)
- **Tables**: `DataTable` (`src/components/data-table/`) + column visibility, sorting, saved views via `globalSavedViewsStore`
- **AI**: `postAi` / `aiApi` for selective features (not all modules)

Module index

#	Module	Doc
1	Lead & Sales	modules/01-lead-sales.md
2	Projects & Inventory	modules/02-projects-inventory.md
3	Booking & Payment	modules/03-booking-payment.md
4	Documents & Compliance	modules/04-documents-compliance.md
5	Procurement (catalog, pricing, capacity, compliance)	modules/05-procurement-management.md
6	Vendor Management	modules/06-vendor-management.md
7	Supplier Management	modules/07-supplier-management.md
8	Work Orders	modules/08-work-orders.md

Cross-module relationships (high level)

```
flowchart LR
  subgraph sales [Lead and Sales]
    L[Leads]
```

```
  C[Customers]
end
subgraph pi [Projects and Inventory]
  P[Projects]
  I[Inventory units]
end
subgraph bp [Booking and Payment]
  B[Bookings]
  Pay[Payments]
  PL[Payment links]
end
subgraph doc [Documents]
  D[Documents]
  E[eSign]
end
subgraph proc [Procurement]
  V[Vendors]
  S[Suppliers]
  WO[Work orders]
  MC[Material catalog]
end
L ⟶ B
P ⟶ I
I ⟶ B
B ⟶ Pay
B ⟶ PL
L ⟶ D
B ⟶ D
V ⟶ WO
S ⟶ MC
WO ⟶ V
```

Shared UX patterns (all modules)

1. **List / hub pages**: dense toolbars (search, filters, export, bulk actions where implemented), optional kanban (leads), pagination.
2. **Record layout**: breadcrumb + tabbed or sectioned **Record** components (`*RecordTabs.tsx`, `*DetailShell.tsx`).
3. **Create / view / edit**:
   - Many entities use **`/ … /view/new`** or **`slug === 'new'`** for create on the same route family as view.
   - **Edit** often uses query param **`?edit=1`** (see `next.config.ts` redirect for leads: `/leads/edit/:slug` → `/leads/view/:slug?edit=1`).
4. **Drafts**: dedicated draft list routes (e.g. booking drafts, work order drafts) + client-side draft services where applicable (`draftService`, booking drafts).

5. **Saved views**: `globalSavedViewsStore` + `useGlobalSavedViewsSync` + route-scoped payloads (e.g. leads filters).
6. **History**: per-module links from sidebar flyouts to `/company-admin/history-logs?module=<id>`; hook `useHistoryLogs` filters `MOCK_HISTORY_LOGS`.

Key source folders

Area	Location
Company-admin routes	`src/app/company-admin/**`
Leads (outside prefix)	`src/app/leads/**`, `src/components/leads/**`
Projects / inventory	`src/app/projects-inventory/**`, `src/components/projects-inventory/**`
Work orders	`src/app/work-orders/**`, `src/components/work-orders/**`
Booking & payment UI	`src/components/booking-payment/**`
Documents & compliance UI	`src/components/documents-compliance/**`
Vendors	`src/components/vendors/**`
Suppliers	`src/components/suppliers/**`
Shared table	`src/components/data-table/**`
History types & mock	`src/lib/historyLogs/**`

Related repo docs

- Root agent guide: `AGENTS.md` (monorepo conventions, stores, Tailwind, edit-mode pattern).

Generated from codebase analysis; routes and components reflect the repository at documentation time.

Module 1 — Lead & Sales

1. Module purpose

Audience	Explanation
Business	Captures demand: prospects (leads), nurtures them through status and assignments, supports intelligence and analytics, and links to **Customers** for company-admin CRM-style views.
Technical	Primary data in `leadStore`; list and record UIs in `components/leads`. Routes mostly live under **`/leads`** (not under `/company-admin`), but the **Company Admin sidebar** exposes them as "Lead & Sales". Shell: `CompanyAdminDashboardLayout` (used inside list page content).
User flow	Land on **Leads** table → open a lead **view** (tabs) → optionally **edit** (`?edit=1`) → related actions (assign, archive, export). Parallel entry: **Create** via `/leads/view/new` or flyout "Create Lead".

Why it exists: Central pipeline for revenue operations before and alongside **bookings** and **projects**.

---

2. Main features

- **Table / list** (`LeadsListPageContent`): pagination, column picker, sorting, filters drawer, bulk selection, bulk status / assign / delete, CSV & print-style export, saved views, import modal (`?import=1`).
- **Kanban** view toggle (same store data, different presentation).
- **Search** debounced / draft search field tied to filter payload.
- **Archived leads** (`/leads/archived`), **drafts** (`/leads/drafts`).
- **Lead record** (`LeadRecordTabs`): multi-tab overview + deep sections (assignment, follow-ups, site visits, pipeline, conversion, broker, notifications, linked bookings/payments, etc. — see component for full tab set).
- **Edit / create**: unified **`/leads/view/[slug]`** with `slug === 'new'` for create; edit query per `next.config` redirect from legacy `/leads/edit/:slug`.
- **Intelligence** (`/leads/intelligence`), **Analytics** (`/leads/analytics`), **AI Demand Intelligence** (`/demand-intelligence` — sidebar under same group).
- **Customers (company admin)** (`/company-admin/customers`, `view/[slug]`, `create`) — CRM list tied to leads/customers store patterns.

---

3. Page structure

Route	Purpose	Main components
`/leads`	Main leads hub	`LeadsListPageContent`, `DataTable`, `LeadsKanbanBoard`, `ImportLeadsModal`
`/leads/view/new`	Create lead (same architecture as view)	`LeadRecordTabs`, `LeadDetailShell`
`/leads/view/[slug]`	View / edit lead	`LeadRecordTabs`, `getLeadBySlugIncludingArchived`
`/leads/create`	May redirect / legacy create path	Check app route
`/leads/drafts`	Draft list	Draft-focused list UI
`/leads/archived`	Archived list	Archive store slice
`/leads/intelligence`	Demand / intel dashboard	Intelligence store + UI
`/leads/analytics`	Analytics	Charts / aggregates
`/leads/[slug]/*`	Feature-specific subpages (site-visit, pipeline, …)	Feature pages
`/demand-intelligence`	AI demand (sidebar sibling)	Demand intelligence module
`/company-admin/customers`	Customer list	`CustomersListPageContent` pattern
`/company-admin/customers/view/[slug]`	Customer dashboard view	`CustomerDashboardView`
`/company-admin/customers/create`	Create customer / handoff	Booking create return URL pattern in codebase

Layouts: No `app/leads/layout.tsx`; pages embed **`CompanyAdminDashboardLayout`** where the full admin chrome is required.

---

4. Table page analysis (Leads list)

- **Columns (data ids):** `name`, `email`, `phone`, `source`, `project`, `budgetRange`, `preferredUnitType`, `status`, `assignedTo`, `createdDate` (+ always `actions`). Defaults defined in `LEADS_TABLE_DEFAULT_ON` in `LeadsListPageContent.tsx`.

- **Filters:** Rich `LeadsFilterPayload` (search, status, source, project, date range, etc.) — see type in same file.
- **Search:** `searchDraft` / `searchTerm` integrated into filter payload.
- **Bulk:** Selected row IDs → bulk assign, bulk status, bulk delete (store functions).
- **Row actions:** `LeadRowActionsMenu` (view, archive, delete, etc.).
- **Pagination:** `ITEMS_PER_PAGE = 10`, `Pagination` component.
- **Export:** `downloadLeadsCsv`, `openLeadsPrintReport`; scope = selection or filtered set.
- **Saved views:** `loadGlobalSavedViews` filtered by normalized pathname; `useConsumePendingSavedView` applies pending payload from navigation; legacy import from `arris-leads-saved-views`.
- **Status:** `LeadStatusBadge`.
- **Import:** `ImportLeadsModal` when `?import=1`.

---

5. View page analysis (Lead record)

- **Shell:** `LeadDetailShell` (breadcrumb, actions, not-found).
- **Tabs:** `LeadRecordTabs` aggregates overview, inline editors, related data, attachments, activity, **linked bookings / payments** (cross-module).
- **Meta / toolbar:** Sticky actions pattern inside tab components (save, cancel) varies by tab.
- **Validation:** Tab-level and overview validators (see `*FormValidation` / inline validation in components).
- **Drafts:** Draft storage patterns where enabled (session/local draft keys — inspect `LeadRecordTabs` and draft hooks if extending).

---

6. Create / edit flow

```
sequenceDiagram
  participant U as User
  participant R as Router
  participant V as Lead view page
  participant T as LeadRecordTabs
  U-->>R: Open /leads/view/new
  R-->>V: slug=new createMode
  V-->>T: Empty draft lead object
  U-->>T: Fill + Save
  T-->>R: Navigate to /leads/view/{slug}
```

- **Same-page create:** `createMode` derived from slug; synthetic draft object built in `page.tsx`.
- **Edit:** Prefer **`?edit=1`** on view URL (global convention in AGENTS.md).
- **Redirects:** Permanent redirect from `/leads/edit/:slug` to view + edit query (`next.config.ts`).

7. History system

- **Module id:** `leads` (see `HISTORY_MODULES` in `historyLogs/types.ts`).
- **Sidebar:** Flyout “Leads history” → `/company-admin/history-logs?module=leads`.
- **Record-level:** Activity / thread data inside lead store (`activityLog`, `threadNotes`) complements global audit mock.

---

8. Relationships

From	To	Mechanism
Leads	Bookings	`linkedBookings` on lead; booking create can prefill `leadCode` query
Leads	Payments	`linkedPayments` display in record
Leads	Projects	`project` field / filters
Leads	Customers	Admin customers views by lead slug
Leads	Documents	Optional linkage via compliance flows (downstream)

---

9. UI / UX patterns

- Enterprise **DataTable** with column visibility popover (`LuColumns3`).
- **Saved views** drawer (`LuBookmark`).
- **Bulk bar** when selection non-empty.
- **Kanban / Table** toggle for pipeline visibility.

---

10. Architecture notes

Topic	Location
Store API	`src/lib/leadStore.ts`
Routes helper	`src/lib/leadRoutes.ts` (`leadProfileHref`, etc.)
CSV export	`src/lib/exportLeadsCsv.ts`
PDF / print	`src/lib/exportLeadsPdf.ts`

Topic	Location
Intelligence store	`src/lib/leadsIntelligenceStore.ts`
Global saved views	`src/lib/globalSavedViewsStore.ts`

Beginner tip: Start at `app/leads/page.tsx` → `LeadsListPageContent` → `app/leads/view/[slug]/page.tsx` → `LeadRecordTabs.tsx` (large but authoritative).

Module 2 — Projects & Inventory Management

1. Module purpose

Audience	Explanation
Business	Defines **projects** (buildings/phases) and **inventory units** (sellable stock), pricing context, visual floor mapping, and analytics — upstream of **bookings** and **payments**.
Technical	Routes under **`/projects-inventory/*`**. Stores: `projectStore`, inventory-related stores (see `lib/*` imports in `ProjectRecordTabs` / list pages). Canvas/visual routes use Konva patterns per AGENTS.md.
User flow	Project list → open **project view** (tabs: overview, units, pricing hooks, etc.) → create/edit units from inventory hub → dashboards / AI insights for portfolio visibility.

---

2. Main features

- **Project list** with filters, saved views, export patterns (see `ProjectsListPageContent` / analogous list components).
- **Project create** `/projects-inventory/projects/create` and **view/new** pattern for unified create.
- **Project drafts** `/projects-inventory/projects/drafts`.
- **Inventory list** `/projects-inventory/inventory`, **create**, **view/[slug]**, **edit/[slug]**.
- **Pricing** hub `/projects-inventory/pricing`.
- **Visual mapping** & **visual inventory view** (canvas-heavy).
- **Inventory dashboard** & **Analytics** & **AI insights** pages.
- **Record tabs** on project: `ProjectRecordTabs` (large component — phases, media, inventory linkage, approvals, etc.).

---

3. Page structure

Route	Purpose
`/projects-inventory/projects`	Project table hub
`/projects-inventory/projects/create`	Create flow entry
`/projects-inventory/projects/view/new`	Alternate unified create URL (sidebar flyout)
`/projects-inventory/projects/view/[slug]`	Project record

Route	Purpose
`/projects-inventory/projects/drafts`	Draft projects
`/projects-inventory/inventory`	Units table
`/projects-inventory/inventory/create`	Add unit
`/projects-inventory/inventory/view/[slug]`	Unit record
`/projects-inventory/inventory/edit/[slug]`	Unit edit
`/projects-inventory/pricing`	Pricing workspace
`/projects-inventory/visual-inventory-mapping`	Visual mapper
`/projects-inventory/visual-inventory-view`	Visual reader
`/projects-inventory/inventory-dashboard`	KPI-style dashboard
`/projects-inventory/analytics`	Analytics
`/projects-inventory/ai-insights`	AI narrative / insights

Shell: Pages compose **`CompanyAdminDashboardLayout`** for sidebar alignment (same as leads).

Sidebar flyouts (`CompanyAdminSidebar.tsx`):

- **Project Setup:** create (→ view/new), drafts, **history** → `?module=projects`.
- **Inventory:** add unit, **history** → `?module=inventory`.

---

4. Table page analysis

- **Projects / inventory lists** follow the shared **DataTable** pattern: sortable columns, filters, optional export (see respective `*ListPageContent.tsx` files under `components/projects-inventory/`).
- **Pagination / empty states:** consistent with leads (per-page limits in each list module).
- **Row actions:** open view, edit slug routes, sometimes inline status.

(Exact column ids vary by list file — inspect `ProjectsListPageContent` / inventory list component for authoritative IDs.)

---

5. View page analysis

- **`ProjectRecordTabs`:** primary project **record** surface; includes overview cards, inventory ties, documents hooks, and workflow sections.

- **Inline editing:** sections use controlled fields + save bars inside tabs.
- **Validation:** project / inventory validation modules in `lib/*FormValidation.ts` (naming pattern).

---

6. Create / edit flow

- **Projects:** Flyout links `projects/create` with **`linkHref` to `projects/view/new`** (same unified create pattern as leads).
- **Inventory:** Dedicated **create** route and **edit/[slug]** for units.
- **Drafts:** `projects/drafts` list recovers in-progress records.

```
flowchart TD
  A[Project list] ⟶ B[view slug or view/new]
  B ⟶ C[ProjectRecordTabs]
  C ⟶ D[Inventory units]
  D ⟶ E[Booking flow later]
```

---

7. History system

- **Modules:** `projects`, `inventory`, `pricing` exist in `HISTORY_MODULES` for global logs.
- **Sidebar links:** `history-logs?module=projects` and `?module=inventory` from flyouts.

---

8. Relationships

From	To	Notes
Projects	Inventory units	Unit records reference project
Inventory	Bookings	Booking captures `unitId` / project name options from `bookingPaymentMockStore.getProjectOptions`
Projects	Pricing	Shared commercial context
Projects	Leads	Lead “project” filter / association

---

9. UI / UX patterns

- **Hub + flyout** navigation for nested actions (create, drafts, history).

- **Konva** for visual mapping pages (imperative canvas — sync carefully with React state).
- **Breadcrumbs** on nested create/edit pages.

---

10. Architecture notes

Topic	Location
Project UI	`src/components/projects-inventory/`
App routes	`src/app/projects-inventory/**`
Stores	`src/lib/projectStore.ts`, inventory stores (grep `inventory` under `lib/`)
Sidebar active rules	`navItemIsActive` for `/projects-inventory/projects` and `inventory` paths — excludes `/create` children where needed

Beginner tip: Open `ProjectRecordTabs.tsx` and search for `tabs` / `TAB` definitions to map the business sections quickly.

Module 3 — Booking & Payment Tracking

1. Module purpose

Audience	Explanation
Business	Converts demand into **bookings** (reservations against units), tracks **payments** and **installments**, issues **payment links**, and surfaces operational **reports**, **system** settings, and **AI** helpers for finance ops.
Technical	Routes live under **`/company-admin/booking-payment/*`** (plus **global history** at `/company-admin/history-logs`). Core mock orchestration in `bookingPaymentMockStore` (and related payment link stores). Wrapped by **`app/company-admin/layout.tsx`** → `CompanyAdminDashboardLayout`.
User flow	Booking hub → create booking (draft-capable) → view booking with tabs → add payments / link documents → payment hub for ledger / installments → payment links for outbound collection.

---

2. Main features

- **Booking list** with hub flyout: create, **drafts**, history (`?module=bookings`).
- **Booking record tabs:** Overview, Payments, Documents, History (`BookingRecordTabs` + `BookingDetailView`).
- **Payments list / add / edit / view / installments** routes.
- **Payment links:** list, form, add, edit, view slug.
- **Reports** page.
- **Booking & Payment AI** dedicated page.
- **System** admin page for module configuration UI.
- **Draft service** integration for long forms (`draftService`, `BookingOverviewTab` draft state).
- **Return navigation** `returnTo` query sanitized via `safeInternalPath`.

---

3. Page structure

Route	Purpose
`/company-admin/booking-payment/booking`	Booking table hub
`/company-admin/booking-payment/booking/create`	Create booking wizard entry

Route	Purpose
`/company-admin/booking-payment/booking/drafts`	Draft bookings
`/company-admin/booking-payment/booking/view/[slug]`	View / create (`new`) / edit (`?edit=1`)
`/company-admin/booking-payment/booking/edit/[slug]`	Redirect-style legacy → view + edit (check `edit/[slug]/page.tsx`)
`/company-admin/booking-payment/payments`	Payments hub
`/company-admin/booking-payment/payments/add`	Add payment (+ `returnTo`)
`/company-admin/booking-payment/payments/edit/[slug]`	Edit payment
`/company-admin/booking-payment/payments/view/[slug]`	Payment detail
`/company-admin/booking-payment/payments/installments`	Installments workspace
`/company-admin/booking-payment/payment-links`	Links list
`/company-admin/booking-payment/payment-links/form`	Link form
`/company-admin/booking-payment/payment-links/add`	Add link
`/company-admin/booking-payment/payment-links/edit/[slug]`	Edit
`/company-admin/booking-payment/payment-links/view/[slug]`	View
`/company-admin/booking-payment/reports`	Reports
`/company-admin/booking-payment/ai`	AI surface
`/company-admin/booking-payment/system`	System
`/company-admin/history-logs?module=bookings`	Scoped history
`/company-admin/history-logs?module=payments`	Scoped history (also linked from payment links flyout)

Key components: `BookingRecordTabs`, `BookingDetailView`, `BookingOverviewTab`, `BookingStepperForm`, payment link components under `components/booking-payment/`.

4. Table page analysis

- **Bookings / payments / links** lists: `DataTable` or module-specific tables with **filters**, **row actions**, **status chips**, **export** where wired.
- **Payment links** standard form: `PaymentLinkFormStandard.tsx` (and variants).
- **Bulk / import:** varies by sub-page — inspect each `page.tsx` under `booking-payment/`.

---

5. View page analysis (Booking record)

Tabs (`TAB_ITEMS` in `BookingRecordTabs.tsx`):

Tab key	Label	Role
`overview`	Overview	Primary fields, draft save, create vs edit vs view
`payments`	Payments	Ledger / payment summaries tied to booking
`documents`	Documents	Linked compliance docs
`history`	History	Change log / timeline for the booking

- **URL tab switching:** `?tab=payments|documents|history` (default overview omits query).
- **Create mode:** `slug === 'new'`; draft keys and `activeDraftId` managed in `BookingRecordTabs`.
- **Edit mode:** `?edit=1` or `?edit=true`.
- **Lead prefill:** `leadCode` query from lead → booking handoff.

---

6. Create / edit flow

```
stateDiagram-v2
  [*] ⟶ List: Booking hub
  List ⟶ Create: create route
  Create ⟶ Draft: Autosave draftService
  Draft ⟶ View: Save booking slug
  View ⟶ Edit: ?edit=1
  Edit ⟶ View: Save clears edit flag
```

- **After save:** `goAfterSave` pushes `returnTo` if present, else `/company-admin/booking-payment/booking/view/{slug}`.
- **Drafts page:** resume from `booking/drafts`.

---

7. History system

- **Modules:** `bookings`, `payments` in `HISTORY_MODULES`.
- **Sidebar flyouts** link to filtered `history-logs`.
- **Tab "History"** on booking for record-scoped narrative (complements global log).

---

8. Relationships

Entity	Connects to
Booking	Lead (`leadId`, `leadCode`), Project/unit options from store
Payment	Booking slug / filters
Payment link	Booking / customer context in forms
Documents tab	Compliance document IDs (read-only / link UI)

---

9. UI / UX patterns

- **Inline toasts:** `InlineToast` for save feedback.
- **Sticky save** on overview during create/edit.
- **Hub row `replaceLink`:** Payments hub clears stale `?booking=` when opened from sidebar (see sidebar comment).

---

10. Architecture notes

Topic	Location
Store	`src/lib/bookingPaymentMockStore.ts`
Drafts	`src/lib/draftService.ts`
Record UI	`src/components/booking-payment/BookingRecordTabs.tsx`
Safe return	`src/lib/navigationReturn.ts`

Security note: Always use **`safeInternalPath`** for `returnTo` to avoid open redirects when extending.

Module 4 — Documents & Compliance

1. Module purpose

Audience	Explanation
Business	Central **document lifecycle** (register, version, classify), **eSign (Aadhaar)** flows, **audit** visibility, **soft-deleted** records, and **AI** assistance for policy-heavy real estate operations.
Technical	Routes under **`/company-admin/documents-compliance`** (+ `documents-compliance-ai`). Primary store: `complianceDocumentsMockStore` (+ RBAC helpers `complianceRbac.ts`). Role hook: `useComplianceRole`.
User flow	Document hub → list / filter → open **view** or **edit** → optional version modal → eSign requests tracked separately.

---

2. Main features

- **Document management hub** `/company-admin/documents-compliance` (table / filters — see page).
- **Unified view** `/company-admin/documents-compliance/view/[id]` with `id === 'new'` for **create**.
- **Edit** routes: `[id]/edit`, `esign/[id]/edit`, etc.
- **eSign** listing + **new** request.
- **Audit & activity** dedicated page.
- **Deleted records** recovery / audit.
- **Documents AI** `/company-admin/documents-compliance-ai`.
- **RBAC:** `canView`, `canEdit` gate buttons and routes.

---

3. Page structure

Route	Purpose
`/company-admin/documents-compliance`	Main hub
`/company-admin/documents-compliance/view/[id]`	View / create (`new`) — `DocumentDetailView`, `DocumentVersionModal`
`/company-admin/documents-compliance/view/new`	Flyout “Add document” target
`/company-admin/documents-compliance/[id]`	Redirect → `/view/[id]`

Route	Purpose
`/company-admin/documents-compliance/[id]/edit`	Redirect to edit flow
`/company-admin/documents-compliance/new`	Legacy / helper redirect
`/company-admin/documents-compliance/esign`	eSign list
`/company-admin/documents-compliance/esign/new`	New eSign
`/company-admin/documents-compliance/esign/[id]/edit`	Edit eSign
`/company-admin/documents-compliance/audit`	Audit
`/company-admin/documents-compliance/deleted`	Deleted bucket
`/company-admin/documents-compliance-ai`	AI

Flyouts (`DOCUMENTS_HUB_FLYOUT`, `ESIGN_HUB_FLYOUT`): quick add, history → `?module=documents`.

---

4. Table page analysis

- **Hub list:** columns for title, type, status, owner, dates (inspect `documents-compliance/page.tsx`).
- **Filters / search:** pattern aligned with other admin tables.
- **Row actions:** view, download (logs `logDownload`), version history.
- **Empty states:** standard card + CTA to upload / create.

---

5. View page analysis

- **`DocumentDetailView`:** tabs driven by `initialTab` query (see `view/[id]/page.tsx` for `tab` parsing).
- **Version modal:** `DocumentVersionModal` for file uploads / version stack.
- **Toolbar:** back to list, edit href when permitted, versions button.

---

6. Create / edit flow

- **Create:** `view/new` route with `createMode` boolean; `getDocumentById` skipped when new.

- **Redirects:** `[id]/page.tsx` uses `router.replace` to canonical view URL — avoids duplicate content routes.
- **Validation:** field-level in forms under `components/documents-compliance/` + schemas in `lib/*` if present.

---

7. History system

- **Module:** `documents` in global history.
- **Audit page:** operational audit separate from `history-logs` mock stream (business-facing).
- **Store-level:** view/download actions logged via `logView`, `logDownload`.

---

8. Relationships

From	To
Documents	Bookings (booking tab shows linked docs)
Documents	Leads / customers
eSign	Document versions
Compliance AI	Policies / doc text

---

9. UI / UX patterns

- **Breadcrumb** stacks with `BASE = '/company-admin/documents-compliance'`.
- **RBAC-driven** disabled buttons vs hidden sections.
- **Modal** for versions to keep main view clean.

---

10. Architecture notes

Topic	Location
Store	`src/lib/complianceDocumentsMockStore.ts`
RBAC	`src/lib/complianceRbac.ts`
UI	`src/components/documents-compliance/*`

Topic	Location
Types	`src/lib/historyLogs/types.ts` (`documents` module)

Beginner tip: Read `view/[id]/page.tsx` first — it composes the main shell and props for `DocumentDetailView`.

Module 5 — Procurement Management (Catalog, Pricing, Capacity, Compliance)

1. Module purpose

Audience	Explanation
Business	Supports **strategic procurement** beyond individual vendor/supplier profiles: **material master data**, **supplier pricing**, **capacity planning**, and **supplier compliance** scoring — aligns purchasing with projects and work execution.
Technical	Routes under **`/company-admin/procurement/*`**. These pages are **first-class** in the **Supplier management** sidebar flyout (see `SUPPLIER_MANAGEMENT_FLYOUT` in `CompanyAdminSidebar.tsx`) alongside supplier CRUD.
User flow	From supplier hub flyout → open catalog / pricing / capacity / compliance → adjust master data → downstream **work orders** and **supplier** records consume context.

> **Note:** "Procurement Management" in the sidebar is a **parent group** that also contains Vendor, Supplier, and Work Orders modules (docs **06–08**). **This file** focuses on the **`/company-admin/procurement/ …`** analytics/catalog surfaces.

---

2. Main features

Feature	Route (typical)
Material master catalog	`/company-admin/procurement/material-catalog`
Supplier pricing	`/company-admin/procurement/supplier-pricing`
Supply capacity	`/company-admin/procurement/supply-capacity`
Supplier compliance dashboard	`/company-admin/procurement/supplier-compliance`

- **Table-heavy** pages with filters (per implementation in each `page.tsx`).
- **Export / KPI cards** where present (inspect each page).

---

3. Page structure

Route	Purpose	Components
… `/material-catalog`	SKU / material rows	Page-local tables + actions
… `/supplier-pricing`	Price lists / tiers	Pricing UI
… `/supply-capacity`	Capacity vs demand	Charts / tables
… `/supplier-compliance`	Compliance checklist / scores	Compliance UI

Shell: Inherits **`app/company-admin/layout.tsx`** (full admin chrome).

Navigation entry: Sidebar → **Procurement Management** → **Supplier management** flyout includes these links (not duplicated on vendor flyout).

---

4. Table page analysis

- Expect **DataTable** or bespoke tables with **column definitions inline** in each page file.
- **Filters:** search + facet filters (verify in source).
- **Row actions:** drill to supplier or material where linking exists.

---

5. View page analysis

- Mostly **single-page analytics** (less multi-tab than leads/bookings). If tabs exist, they are local `useState` tab bars in the page component.

---

6. Create / edit flow

- Typically **inline edit** or **modal** create for catalog rows (check `material-catalog/page.tsx` for patterns).
- No global `?edit=1` convention here unless a record view was added later — prefer reading the page component.

---

7. History system

- Global history modules include **`suppliers`**; procurement operations may log under suppliers or future `procurement` module if backend extends `HISTORY_MODULES`.
- Today: use **`/company-admin/history-logs?module=suppliers`** for supplier-related audit when wiring events.

---

8. Relationships

```
flowchart TD
  MC[Material catalog]
  SP[Supplier pricing]
  SC[Supplier compliance]
  SU[Supplier profiles]
  WO[Work orders]
  MC ⟶ SU
  SP ⟶ SU
  SC ⟶ SU
  SU ⟶ WO
```

---

9. UI / UX patterns

- Consistent **company** blue styling (`variant="company"` buttons).
- **Breadcrumbs** optional; many procurement pages use page header only.

---

10. Architecture notes

Topic	Location
App routes	`src/app/company-admin/procurement/**`
Supplier linkage	`src/lib/suppliers/*`, `components/suppliers/*`
Cross-links from companies UI	`companies/page.tsx` materials tile → `material-catalog` (post-refactor)

Beginner tip: Cross-check **`CompanyAdminSidebar.tsx`** `SUPPLIER_MANAGEMENT_FLYOUT` whenever you add a procurement route so the flyout stays authoritative for discoverability.

Module 6 — Vendor Management

1. Module purpose

Audience	Explanation
Business	Manages **vendor master data**, **assignments** to jobs/projects, **compliance** documentation, **contracts**, **performance** KPIs, and **alerts** — feeds **work orders** and procurement risk controls.
Technical	Routes under **`/company-admin/vendors`** (+ subpaths). UI in `src/components/vendors/`. Sidebar hub row + **`VENDOR_MANAGEMENT_FLYOUT`**.
User flow	Vendor list → open vendor profile / detail → manage compliance & contracts → monitor alerts and performance.

---

2. Main features

Flyout shortcuts (`CompanyAdminSidebar.tsx`):

- Create vendor
- Vendor list
- Vendor assignments
- Vendor compliance
- Vendor contracts
- Vendor performance
- Vendor alerts center
- **History** → `/company-admin/history-logs?module=vendors`

List / profile patterns: `VendorListPage`, `VendorProfilePage`, specialized centers (`VendorContractsCenterPage`, etc. — discover via `components/vendors/`).

---

3. Page structure

Route	Purpose
`/company-admin/vendors`	Hub / list entry
`/company-admin/vendors/list`	Explicit list (also treated as active when on `/vendors` root per `flyoutEntryIsActive`)

Route	Purpose
`/company-admin/vendors/create`	Create vendor (layout uses **full-bleed** main in `company-admin/layout.tsx`)
`/company-admin/vendors/[id]`	Vendor profile / detail
`/company-admin/vendors/assignments`	Assignments matrix / table
`/company-admin/vendors/compliance`	Compliance
`/company-admin/vendors/contracts`	Contracts center
`/company-admin/vendors/performance`	Performance analytics
`/company-admin/vendors/alerts`	Alerts

---

4. Table page analysis

- **Vendor list:** sortable columns, filters, row link to `[id]`.
- **Assignments:** likely dual-axis (vendor × work area) — inspect `assignments/page.tsx`.
- **Alerts / performance:** KPI tiles + tables.

---

5. View page analysis

- **Vendor profile** uses **tabs** (`VendorMainTabBar`, `supplierDetailTabIds`-style patterns — confirm in `VendorProfilePage.tsx`).
- **Inline editing** on profile sections with save/cancel per section or global save.

---

6. Create / edit flow

- **Create:** `/company-admin/vendors/create` — special-cased layout removes max-width for long forms.
- **Edit:** typically embedded in profile tabs or dedicated edit routes inside `[id]` subtree (grep `edit` under `vendors/`).

---

7. History system

- **Module:** `vendors` in `HISTORY_MODULES`.
- **Sidebar:** “Vendors history” flyout link.
- **Profile:** optional `history` tab or embedded timeline in vendor components.

---

8. Relationships

From	To
Vendors	Work orders
Vendors	Contracts / compliance
Vendors	Company admin dashboard

---

9. UI / UX patterns

- **Flyout navigation** mirrors Booking hub pattern.
- **Full-bleed create** for dense vendor onboarding forms.
- **DataTable** with persisted column state (`storageKey` pattern in vendor tables).

---

10. Architecture notes

Topic	Location
Components	`src/components/vendors/**`
Stores	`src/lib/vendors*` or `vendor*` (glob `lib` for vendor)
Sidebar active logic	`navItemIsActive` excludes `/create`, `/assignments`, etc., for hub row vs child highlighting

Beginner tip: Start at `app/company-admin/vendors/page.tsx` and follow imports into `VendorListPage` / `VendorProfilePage`.

Module 7 — Supplier Management

1. Module purpose

Audience	Explanation
Business	Maintains **supplier** relationships distinct from **vendors** (often materials / logistics vs services), including **list/create**, **profile**, and ties to **procurement** catalog and **compliance** artifacts.
Technical	Routes **`/company-admin/suppliers`** (+ `list`, `create` redirect, `[id]`). Components under `src/components/suppliers/`. Store stack in `src/lib/suppliers/`. Sidebar flyout: **`SUPPLIER_MANAGEMENT_FLYOUT`**.
User flow	Supplier hub → list → open supplier **profile** with tabs → jump to procurement satellite pages (catalog, pricing, capacity, compliance).

---

2. Main features

Flyout entries:

- Create supplier (`/company-admin/suppliers/create` → redirects to **`/company-admin/suppliers/new`** with query preserved)
- Supplier list
- Material Catalog (procurement)
- Supplier Pricing (procurement)
- Supply Capacity (procurement)
- Supplier Compliance (procurement)
- **History** → `?module=suppliers`

Profile UI: `SupplierProfilePage` with tab bar and data tables per tab.

---

3. Page structure

Route	Purpose
`/company-admin/suppliers`	Hub
`/company-admin/suppliers/list`	List (active state merged with hub root in sidebar helper)
`/company-admin/suppliers/create`	Redirect to … `/new`

Route	Purpose
`/company-admin/suppliers/new`	Actual create surface (layout full-bleed like vendors create)
`/company-admin/suppliers/[id]`	Supplier profile

---

4. Table page analysis

- **Supplier list:** similar to vendors — `DataTable`, filters, link to profile.
- **Tab tables inside profile:** materials, pricing, bundles — see `SupplierProfilePage.tsx`.

---

5. View page analysis

- **Tabs:** `SupplierMainTabBar` + tab ids in `supplierDetailTabIds.ts`.
- **Create mode:** `createMode` flag inside profile when `id` is `new` (pattern mirrors other modules).

---

6. Create / edit flow

```
flowchart LR
  C[/suppliers/create] ⟶ N[/suppliers/new]
  N ⟶ P[SupplierProfilePage createMode]
  P ⟶ S[Save persists to supplierRelationsStore]
```

- **Redirects:** `create/page.tsx` uses `router.replace` to canonical `new` route.

---

7. History system

- **Module:** `suppliers` in `HISTORY_MODULES`.
- **Sidebar:** “Suppliers history” link from flyout.

---

8. Relationships

From	To
Suppliers	Material catalog

From	To
Suppliers	Work orders
Suppliers	Company pages

---

9. UI / UX patterns

- **Tabbed profile** with **DataTable** per tab and persisted preferences (`storageKey` suffix `-supplier-*-tab-v1`).
- **Modal editing** for nested rows (materials) in `SupplierProfilePage`.

---

10. Architecture notes

Topic	Location
UI	`src/components/suppliers/**`
Store / mock	`src/lib/suppliers/*`
Types	`src/lib/suppliers/types.ts`

Beginner tip: Compare **vendor** vs **supplier** create layouts — both opt into **full-bleed** `CompanyAdminLayout` branch for long forms.

Module 8 — Work Orders

1. Module purpose

Audience	Explanation
Business	Tracks **operational work** assigned to vendors/suppliers: scope, status, cost, schedules, and **field execution** details — connects procurement to delivery.
Technical	Routes under **`/work-orders`** (root app, not under `/company-admin` prefix) with **`CompanyAdminDashboardLayout`** on pages for consistent chrome. Store: `workOrderStore` (`getWorkOrderBySlugIncludingArchived`, `buildEmptyWorkOrderDraft`). Primary UI: `WorkOrderRecordTabs`, `WorkOrderDetailShell`.
User flow	List → open WO **view** → tabbed detail / inline edits → **drafts** list for in-progress WOs → **create** at `/work-orders/view/new`.

---

2. Main features

- **List page** `/work-orders` — table with filters, bulk actions (if enabled), link to view.
- **View / create** `/work-orders/view/[slug]` with `slug === 'new'` for **create** (canonical).
- **Legacy create redirect** `/work-orders/create` → **`/work-orders/view/new`** (`create/page.tsx`).
- **Drafts** `/work-orders/drafts`.
- **Flyout:** create, drafts, **history** → `?module=work_orders`.

---

3. Page structure

Route	Purpose
`/work-orders`	List hub
`/work-orders/view/new`	Create (empty draft from `buildEmptyWorkOrderDraft`)
`/work-orders/view/[slug]`	Detail / edit tabs
`/work-orders/drafts`	Draft picker
`/work-orders/create`	Redirect only

Components: `WorkOrderDetailShell` (breadcrumb, layout), `WorkOrderRecordTabs` (tabs, inline overview editor, etc.).

4. Table page analysis

- **Columns:** work order id, title, vendor, status, dates — confirm in `work-orders/page.tsx` / list content component.
- **Filters:** status / vendor / search (per implementation).
- **Row actions:** open `view/[slug]`.
- **Pagination:** standard `DataTable` / `Pagination` if wired.

---

5. View page analysis

- **Breadcrumbs:** set in `view/[slug]/page.tsx` — includes “Procurement Management” label for business context.
- **Tabs:** `WorkOrderRecordTabs` — includes overview, line items / costs, attachments, history (inspect file for authoritative tab ids).
- **Local persistence:** `activeWorkOrderSlug` saved to `localStorage` on slug change (for refresh / cross-page continuity).
- **Not found:** `WorkOrderNotFound` component when slug missing.

---

6. Create / edit flow

Mode	Detection
Create	`slug === 'new'`
View	existing slug, no edit query
Edit	sections inside tabs toggle edit state or use shared inline editor (`WorkOrderInlineOverviewEditor`)

Store bump: `listVersion` / `onBump` pattern refreshes after mutations.

---

7. History system

- **Module:** `work_orders` in `HISTORY_MODULES`.
- **Sidebar:** “Work orders history” → filtered global log.
- **Record tabs:** likely dedicated history subview inside `WorkOrderRecordTabs`.

---

8. Relationships

From	To
Work orders	Vendors
Work orders	Suppliers / materials
Work orders	Projects

---

9. UI / UX patterns

- **Detail shell** wraps tabs with consistent padding and breadcrumb.
- **Inline editors** for quick field updates on overview tab.
- **Drafts** first-class route (same philosophy as booking drafts).

---

10. Architecture notes

Topic	Location
Pages	`src/app/work-orders/**`
Components	`src/components/work-orders/**`
Store	`src/lib/workOrderStore.ts` (name may vary slightly — grep `workOrder`)
Sidebar hub	`workOrdersHubRowIsActive` matches any `/work-orders` prefix + history module

Beginner tip: Read `view/[slug]/page.tsx` (~40 lines) then jump to **`WorkOrderRecordTabs.tsx`** — it is large but is the single source of truth for WO UX.